# case4.java

```
package com.example;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.web.bind.annotation.*;

/**
 * Case Study 4: Employee Salary Calculator
 *
 * Problem Statement:
 * An organization wants to calculate an employee's monthly salary based on
 * basic salary, allowances, and deductions.
 *
 * Concepts: Dependency injection, Service, REST API, business logic
 *
 * Requirements:
 * 1. Accept employee name and basic salary.
 * 2. Calculate allowances as 20% of basic salary.
 * 3. Calculate deductions as 10% of basic salary.
 * 4. Return the net salary.
 *
 * Formula:
 * - Allowances = basicSalary * 0.20 (20%)
 * - Deductions = basicSalary * 0.10 (10%)
 * - Net Salary = basicSalary + Allowances - Deductions
 */

@SpringBootApplication
public class case4 {

    public static void main(String[] args) {
        SpringApplication.run(case4.class, args);
        System.out.println("Employee Salary Calculator is running on http://localhost:8080");
    }
}

// ---------------- MODEL CLASS ----------------
class Employee {
    private String name;
    private double basicSalary;
    private double allowances;
    private double deductions;
    private double netSalary;

    // Default constructor
    public Employee() {
    }

    // Constructor with name and basicSalary
    public Employee(String name, double basicSalary) {
        this.name = name;
        this.basicSalary = basicSalary;
    }

    // Getters and Setters
    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public double getBasicSalary() {
        return basicSalary;
    }

    public void setBasicSalary(double basicSalary) {
        this.basicSalary = basicSalary;
    }

    public double getAllowances() {
        return allowances;
    }

    public void setAllowances(double allowances) {
```

```
        this.allowances = allowances;
    }

    public double getDeductions() {
        return deductions;
    }

    public void setDeductions(double deductions) {
        this.deductions = deductions;
    }

    public double getNetSalary() {
        return netSalary;
    }

    public void setNetSalary(double netSalary) {
        this.netSalary = netSalary;
    }

    @Override
    public String toString() {
        return "Employee [name=" + name + ", basicSalary=" + basicSalary +
               ", allowances=" + allowances + ", deductions=" + deductions +
               ", netSalary=" + netSalary + "]";
    }
}

// ---------------- SERVICE CLASS (Business Logic) ----------------
@Service
class SalaryService {

    // Business logic to calculate monthly salary components
    public Employee calculateMonthlySalary(Employee employee) {
        double basic = employee.getBasicSalary();

        // 1. Calculate allowances as 20% of basic salary
        double allowances = basic * 0.20;

        // 2. Calculate deductions as 10% of basic salary
        double deductions = basic * 0.10;

        // 3. Calculate net salary
        double netSalary = basic + allowances - deductions;

        // Set the calculated values in employee object
        employee.setAllowances(allowances);
        employee.setDeductions(deductions);
        employee.setNetSalary(netSalary);

        return employee;
    }
}

// ---------------- REST CONTROLLER ----------------
@RestController
@RequestMapping("/salary")
class SalaryController {

    // Demonstrating Dependency Injection (@Autowired)
    @Autowired
    private SalaryService salaryService;

    // Endpoint 1: Calculate monthly salary via JSON body (POST)
    // URL: POST http://localhost:8080/salary/calculate
    /*
     * Sample Request Body:
     * {
     *     "name": "Priya Sharma",
     *     "basicSalary": 50000.0
     * }
     *
     * Expected Response:
     * {
     *     "name": "Priya Sharma",
     *     "basicSalary": 50000.0,
     *     "allowances": 10000.0,
     *     "deductions": 5000.0,
     *     "netSalary": 55000.0
     * }
     */
    @PostMapping("/calculate")
    public Employee calculateSalary(@RequestBody Employee employee) {
```

```
        return salaryService.calculateMonthlySalary(employee);
    }

    // Endpoint 2: Calculate monthly salary via Query Parameters (GET)
    // URL: GET http://localhost:8080/salary/calculate?name=Priya&basicSalary=50000
    @GetMapping("/calculate")
    public Employee calculateSalaryGet(@RequestParam String name, @RequestParam double basicSalary) {
        Employee employee = new Employee(name, basicSalary);
        return salaryService.calculateMonthlySalary(employee);
    }
}
```